Using the Golang “list” Datatype to Implement a Parse Stack for an SR Parser

CMPS 450

Dr. Maida

The “container/list” Package Implements a Doubly Linked List

1. To use: import “container/list”
2. Documentation at: https://golang.org/pkg/container/list/
3. We implement a parse stack for an SR parser using this package.
4. The data structure is named **parseStack**.
5. **parseStack** contains entries of data structure **pstackItem**.
6. These data structures are defined next.

Defining “pstackItem”

parse stack entries

```
type pstackItem struct {
    grammarSym, stateSym string
}
```

1. Use a struct to implement an item in an SR parse stack.
2. An instance of pstackItem has two fields.
3. Both fields are strings.
4. The first field holds a grammar symbol and the second holds a string representation of a state id which is an integer.

Defines format used for printing

```
func (se pstackItem) String() string {
    return se.grammarSym + se.stateSym
}
```

1. Define the String() method.
2. Print statements in Go recognize this so they know how to print a pstackItem.
3. To match the textbook, we concatenate the two fields together to get the output string.

The struct datatype gives us all the operations needed to perform on a “pstackItem”: accessing fields and building new instances.

Aside: defining the method “String() string” implicitly implements the interface Stringer

We defined “pstackItem”, now define “parseStack”

```
type parseStack struct {
    stack *list.List
}
```

1. A “parseStack” is a struct with one field.
2. It’s easier to get call-by-ref to work by using a struct field (shallow copy).
3. The field is a pointer to a “List” in package “list”.
4. The List can hold any datatype, but this list only holds “pstackItems”.

```
func newParseStack() parseStack {
    ps := parseStack{}
    ps.stack = list.New()
    return ps
}
```

newParseStack

1. This function builds and returns a new empty parseStack.
2. It initializes the “stack” field to an empty list.

```
func (stk parseStack) String() string
```

String()

1. The String() method is defined in the sample code.
2. It converts a parseStack to the string representation from the textbook.

“parseStack” Interface

In addition to the previous slide, the “parseStack” datatype implements the following methods. These make sense for the SR application.

1. push(itm pstackItem): pushes a “pstackItem” onto the “parseStack”.

2. top(): returns the item on top of the “parseStack” without changing the stack.

3. pop(): pops the top item from the “parseStack” and also returns the item.

4. popNum(n int): pops n items from “parseStack”. This helps implement the reduce operation, which will pop r items from the stack.

Illustrative Example: Implementing Pop()

Type definition repeated from earlier.

```
type parseStack struct {
    stack *list.List
}
```

1. The stack field in parseStack is a List, but the datatype of the stack elements is not explicitly specified in the declaration.
2. We know the elements will be of type pstackItem.
3. With this in mind, look at the implementation of pop().

Implementation of pop()

```
func (stk parseStack) pop() pstackItem {
    e := stk.stack.Front()
    if e != nil {
        stk.stack.Remove(e)
        return e.Value.(pstackItem)
    }
    return pstackItem{"", ""}
}
```

If the stack is empty, return an empty pstackitem.

1. The method Front() returns the element at the top of the stack and stores it in variable “e”.
2. Front() is part of the “list/List” API.
3. If the stack is not empty, we remove this element from the stack.
4. Next, we return the value of “e”.
5. Notice that we use a type assertion. We have to tell the system that this is a pstackItem b/c it was not declared in parseStack.
6. This is similar to down casting in Java, because the stack item is assumed to be of type, object, and we type cast it to a class lower in the hierarchy.